UNIT–V

Turing machines (TM): Basic model, definition and representation, Instantaneous Description, Language acceptance by TM, Variants of Turing Machine, TM as Computer of Integer functions, Universal TM, Church's Thesis, Recursive, and recursively enumerable languages, Halting problem, Introduction to undecidability, undecidable problems about TM, NP-hard and NP-complete problem, Post correspondence problem (PCP), Modified PCP, Introduction to recursive function theory.

// By Manish D@nge

Turing Machine in TOC

Turing Machines (TM) play a crucial role in the Theory of Computation (TOC). They are abstract computational devices used to explore the limits of what can be computed. Turing Machines help prove that certain languages and problems have no algorithmic solution. Their simplicity makes them an effective tool for studying computational theory, yet they are as powerful as modern computers. Turing Machine was invented by Alan Turing in 1936 and it is used to accept Recursive Enumerable Languages (generated by Type-0 Grammar).

- A Turing Machine consists of an infinite tape, a read/write head, and a set of rules that determine how it reads, writes, and moves on the tape. Despite its simplicity, a TM can simulate any real-world computation, making it as powerful as modern computers but with infinite memory.
- The Turing machine's behavior is determined by a finite state machine, which consists of a finite set of states, a transition function that defines the actions to be taken based on the current state and the symbol being read, and a set of start and accept states.
- The TM begins in the start state and performs the actions specified by the transition function until it reaches an accept or reject state. If it reaches an accept state, the computation is considered successful; if it reaches a reject state, the computation is considered unsuccessful.
- In the context of automata theory and the theory of computation, Turing machines are used to study the properties of algorithms and to determine what problems can and cannot be solved by computers. They provide a way to model the behavior of algorithms and to analyze their computational complexity, which is the amount of time and memory they require to solve a problem.

Why Turing Machines?

Turing machines are an important tool for studying the limits of computation and for understanding the foundations of computer science. They provide a simple yet powerful model of computation that has been widely used in research and has had a profound impact on our understanding of algorithms and computation.

While one might consider using programming languages like C to study computation, Turing Machines are preferred because:

- They are simpler to analyze.
- They provide a clear, mathematical model of computation.
- They possess infinite memory, making them even more powerful than real-world computers.

A Turing Machine consists of a tape of infinite length on which read and writes operation can be performed. The tape consists of infinite cells on which each cell either contains input symbol or a special symbol called blank. It also consists of a head pointer which points to cell currently being read and it can move in both directions.

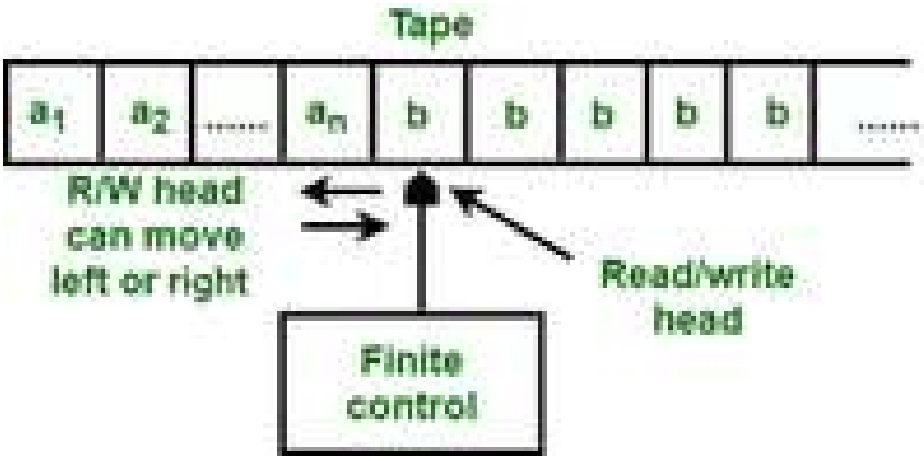


Figure: Turing Machine

Turing Machine Formalism

A Turing Machine is defined by:

1. A finite set of states (Q).
2. An input alphabet (Σ).
3. A tape alphabet (Γ) that includes Σ.
4. A transition function (δ).
5. A start state (q0).
6. A blank symbol (B).
7. A set of final states (F).

The Turing Machine is the basic fundamental model of a modern computer. It is an abstract model of computation. It was proposed by Alan Turing in 1936. At that time, there were no computers. The Turing machine can carry out any computational process that a modern computer can perform. It is the machine format of unrestricted language.

In this chapter, we will cover the basic concepts of Turing machine with its representation and examples of how it works.

Basics of Turing Machine

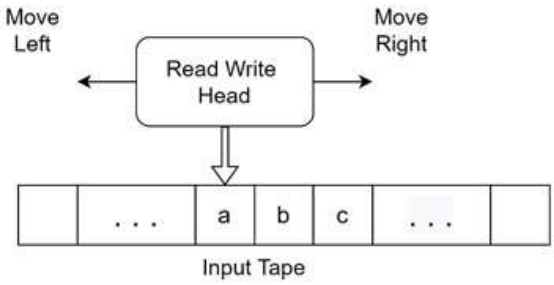
A Turing machine (TM) is defined by 7 tuples: (Q, Σ, Γ, δ, q0, B, F)

- **Q** – Finite set of states
- **Σ** – Finite set of input alphabets
- **Γ** – Finite set of allowable tape symbols
- **δ** – Transitional function
- **q0** – Initial state
- **B** – A symbol of Γ called blank
- **F** – Final state

The transitional function δ is a mapping from Q × Γ → (Q Γ × {L, R, H}). This means that from one state, by getting one input from the input tape, the machine moves to a state. It writes a symbol on the tape and moves to left, right, or halts.

Mechanical Diagram of Turing Machine

A Turing machine consists of three main parts: an input tape, a read-write head, finite control.



The input tape contains the input alphabets. It has an infinite number of blanks at the left and right side of the input symbols. The read-write head reads an input symbol from the input tape and sends it to the finite control. The machine must be in some state. In the finite control, the transitional functions are written. Based on the present state and the present input, a suitable transitional function is executed.

Operations of Turing Machine

When a transitional function is executed, the Turing machine performs these operations –

- The machine goes into some state.
- The machine writes a symbol in the cell of the input tape from where the input symbol was scanned.
- The machine moves the reading head to the left or right or halts.

Instantaneous Description (ID) of Turing Machine

The Instantaneous Description (ID) of a Turing machine remembers the following at a given instance of time –

- The contents of all the cells of the tape, starting from the rightmost cell up to at least the last cell, containing a non-blank symbol and containing all cells up to the cell being scanned.
- The cell currently being scanned by the read-write head.
- The state of the machine.

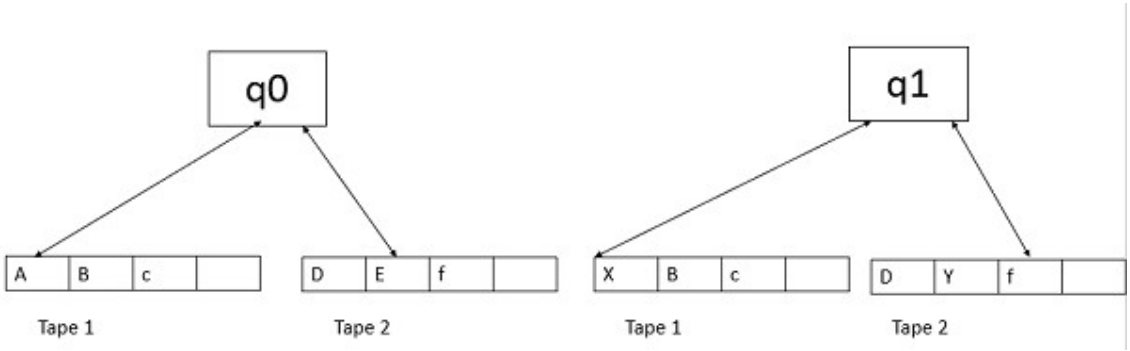
Variations of Turing Machine

Turing machines are powerful computational models that can simulate any algorithmic process. A standard Turing machine consists of a single tape and a single read-write head. However, there are variations of Turing machine that have been developed to address different computational challenges. These variations differ mainly in structure and operation, but they all have the same computational power as the standard Turing machine.

In this chapter, we will present an overview of these different types of Turing machines with diagrams for a better understanding.

Multi-tape Turing Machine

As the name suggests, a multi-tape Turing machine is an extension of the standard Turing machine where multiple tapes are available for input, output, and computation. Each tape has its own read-write head, and the machine's transition function is based on the current state and the symbols read by each head.

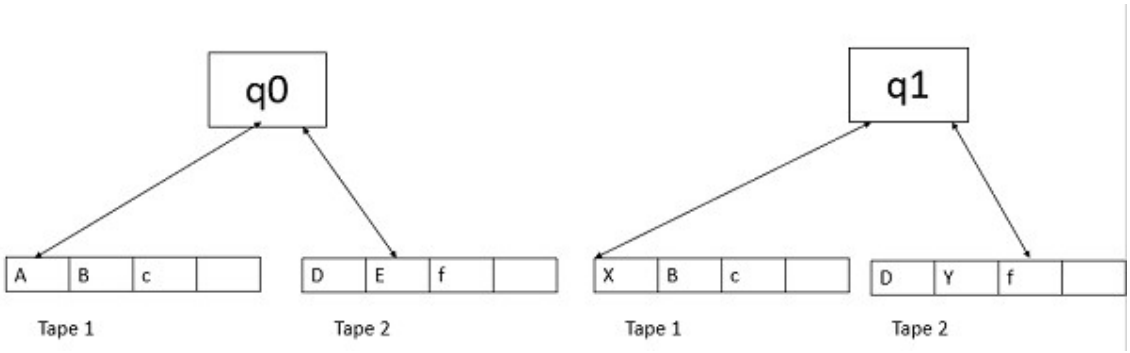


Example

Consider the problem of checking if a binary string is a palindrome or not. A two-tape Turing machine can solve this by copying the string from the first tape to the second, then comparing the string on both tapes in opposite directions.

The process involves copying input from one tape to another, tracing the first tape from left to right while moving the second tape from left to right, and comparing symbols. If all corresponding symbols match, the string is a palindrome, reducing time complexity compared to a single-tape machine.

Multi-tape Turing Machines have multiple tapes where each tape is accessed with a separate head. Each head can move independently of the other heads. Initially the input is on tape 1 and others are blank. At first, the first tape is occupied by the input and the other tapes are kept blank. Next, the machine reads consecutive symbols under its heads and the TM prints a symbol on each tape and moves its heads.



A Multi-tape Turing machine can be formally described as a 6-tuple (Q, X, B, δ, q_0, F) where –

- **Q** is a finite set of states
- **X** is the tape alphabet
- **B** is the blank symbol
- **δ** is a relation on states and symbols where

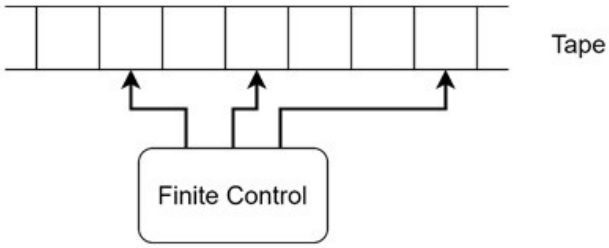
$\delta: Q \times X^k \rightarrow Q \times (X \times \{Left\_shift, Right\_shift, No\_shift\})^k$

where there is **k** number of tapes

- $\mathbf{q_0}$ is the initial state
- **F** is the set of final states

Multi-head Turing Machine

In a multi-head Turing machine, a single tape is used, but it has multiple read-write heads. These heads can independently read and write symbols, enabling the machine to perform complex tasks more efficiently.



Example

The same palindrome checking can be done with this also. The heads start at both ends of the string and move towards each other, checking if the corresponding symbols are identical.

- If multiple heads attempt to write different symbols to the same cell, a priority system determines which head's action is executed.
- If a head tries to move left beyond the leftmost cell, a special condition known as "hanging" occurs, which needs to be handled.
- The multi-head Turing machine, like the multi-tape machine, can be converted to an equivalent single-head machine, although the process may be more complex.

A standard Turing machine has a single read-write head that interacts with an infinitely long tape. However, there is a variation of Turing machine that features multiple read-write heads on a single tape, allowing it to perform tasks more efficiently by processing multiple parts of the tape simultaneously. It is called a multi-head Turing machine.

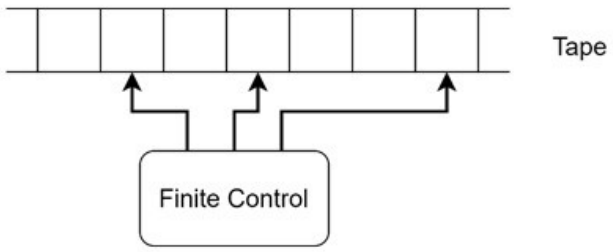
In this chapter, we will explore the concept of Multi-head Turing Machine in detail with examples for a better understanding.

Basics of Multi-head Turing Machine

A Multi-head Turing Machine is similar to the standard Turing machine but with an important modification –

- It has multiple heads instead of just one.
- These heads are connected to a single finite control unit, which directs the machine's operations.
- Each head can read and write symbols independently, enabling the machine to perform more complex operations in parallel.

The functional block diagram of the machine is looking like below –



Key Characteristics of Multi-head Turing Machines

Let us see some of the important characteristics of the multi-head Turing machines –

- **Multiple Heads** – The Turing machine can have several read-write heads, each of which can move independently across the tape.
- **Single Tape** – Unlike the multi-tape Turing machine, the multi-head Turing machine still operates on a single tape.
- **Transition Function** – The transition function in a multi-head Turing machine is more complex. It takes into account the symbols under each head and determines the next state, the symbols to write, and the movements of each head.

Why Use Multiple Heads?

Why do we have to use multi-head Turing Machines? The primary advantage of using multiple heads is that it allows the machine to process different parts of the tape simultaneously. This can significantly reduce the time complexity of certain computational tasks, making the multi-head Turing machine more efficient than its single-head counterpart.

How Does a Multi-head Turing Machine Work?

In a multi-head Turing machine, each head operates independently but is controlled by the same finite control. The transition function defines how the machine reacts based on the current state and the symbols read by all the heads.

Transition Function

The transition function for a multi-head Turing machine can be expressed as –

$$\delta(Q,\Sigma 1,\Sigma 2,\ldots,\Sigma n)\rightarrow(Q,(\Gamma 1\times\{L,R,N\}),(\Gamma 2\times\{L,R,N\}),\ldots,(\Gamma n\times\{L,R,N\}))$$

Where,

- **Q** is the current state.
- $\boldsymbol{\Sigma_1, \Sigma_2, \ldots, \Sigma_n}$ are the symbols read by each head.
- $\boldsymbol{\Gamma_1, \Gamma_2, \ldots, \Gamma_n}$ are the symbols to be written by each head.
- **L, R, N** indicate the direction of movement (Left, Right, No movement) for each head.

TM as Computer of Integer functions:

Turing Machines (TMs) can effectively compute integer functions by representing in tegers in unary format and performing operations like addition and subtraction thro ugh systematic state transitions and tape manipulations.

Overview of Turing Machines

A Turing Machine is a theoretical model of computation that consists of:

- **Tape**: An infinitely long strip divided into cells, each capable of holding a symbol (like 0 or 1) or being blank.
- **Head**: A device that reads and writes symbols on the tape and can move left or ri ght.
- **Finite Control**: A state machine that determines the next action based on the cu rrent state and the symbol under the head.

Integer Representation

In the context of Turing Machines, integers can be represented in **unary format**. For exa mple, the integer 3 can be represented as "000" (three 0s). When performing operations on two integers, they can be separated by a blank symbol (B). For instance, "000B00" re presents the numbers 3 and 2.

Computing Integer Functions

1. **Addition**: To add two integers using a Turing Machine:

- Place the two numbers on the tape, separated by a blank.
- The TM reads the first number and replaces each 0 with a marker (like X) while m oving to the right.
- When it encounters the blank, it writes a 0 in its place and continues until all 0s o f the first number are processed.
- The result is written on the tape in unary format.

2. **Subtraction**: The process is similar to addition:
    - The TM will read the larger number and remove 0s from it while marking the small er number until it reaches the end of the smaller number.
    - The remaining 0s on the tape represent the result of the subtraction.
3. **Multiplication**: This can be achieved by repeated addition:
    - The TM copies the first number to a new location on the tape for each 0 in the sec ond number, effectively adding the first number to itself multiple times.

Conclusion

Turing Machines are powerful computational models that can simulate any algorithm, i ncluding those for integer functions. By using systematic tape manipulations and state t ransitions, TMs can perform basic arithmetic operations on integers represented in unar y format, demonstrating their capability as universal computers for integer functions.

Universal Turing Machine

The UTM was associated with Alan Turing back in 1936. Being a theoretical device, it is able to run any Turing machine on its tape, an idea that represents general computation. The idea created an awakening of what could be done computationally, algorithmically, and by machines. Because a single machine can execute any algorithm, the UTM truly models the heart of modern computing systems and at the same time explores the boundaries of what can be computed.

What is a Universal Turing Machine (UTM)?

A Universal Turing Machine can be defined as a theoretical construction that can simulate the behavior of other machines. This is done specifically by reading, apart from the input tape it simulates, a description of how this very machine it is simulating would work, and transition rules showing how it would respond to the input. This gives the UTM the character of a general-purpose computational device that can execute any arbitrary computable function. Universal Turing machines have hence been taken to provide the means for the Church-Turing thesis: anything computable can be computed by a Turing machine. In this sense, the UTM is more properly considered to be a universal interpreter that can run any algorithm specified by another machine.

A Universal Turing Machine is a Turing Machine which when supplied with an appropriate description of a Turing Machine M and an input string w, can simulate the computation of w.

Universal Turing Machine

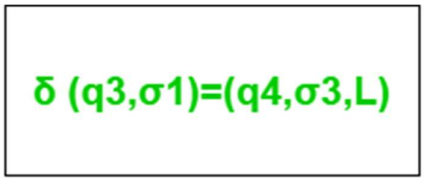
Construction of UTM

Without loss of generality, we assume the following for M:

- Q = {q1, q2, ...qn} where q1=initial state and q2=Final State

- τ = {σ1, σ2,,...σn} where σ represent blanks
- Select an encoding on which q1 is representable by 1, q2 by 11, and so on.
- Similarly, σ1 is encoded as 1, σ2 as 11, etc.
- Finally, let us represent R/W head directions by 1 for L (Left) and 11 for R(Right).
- The symbol 0 will be used as a separator between 1s.

With this scheme, any transition of M can be given as :



will appear as

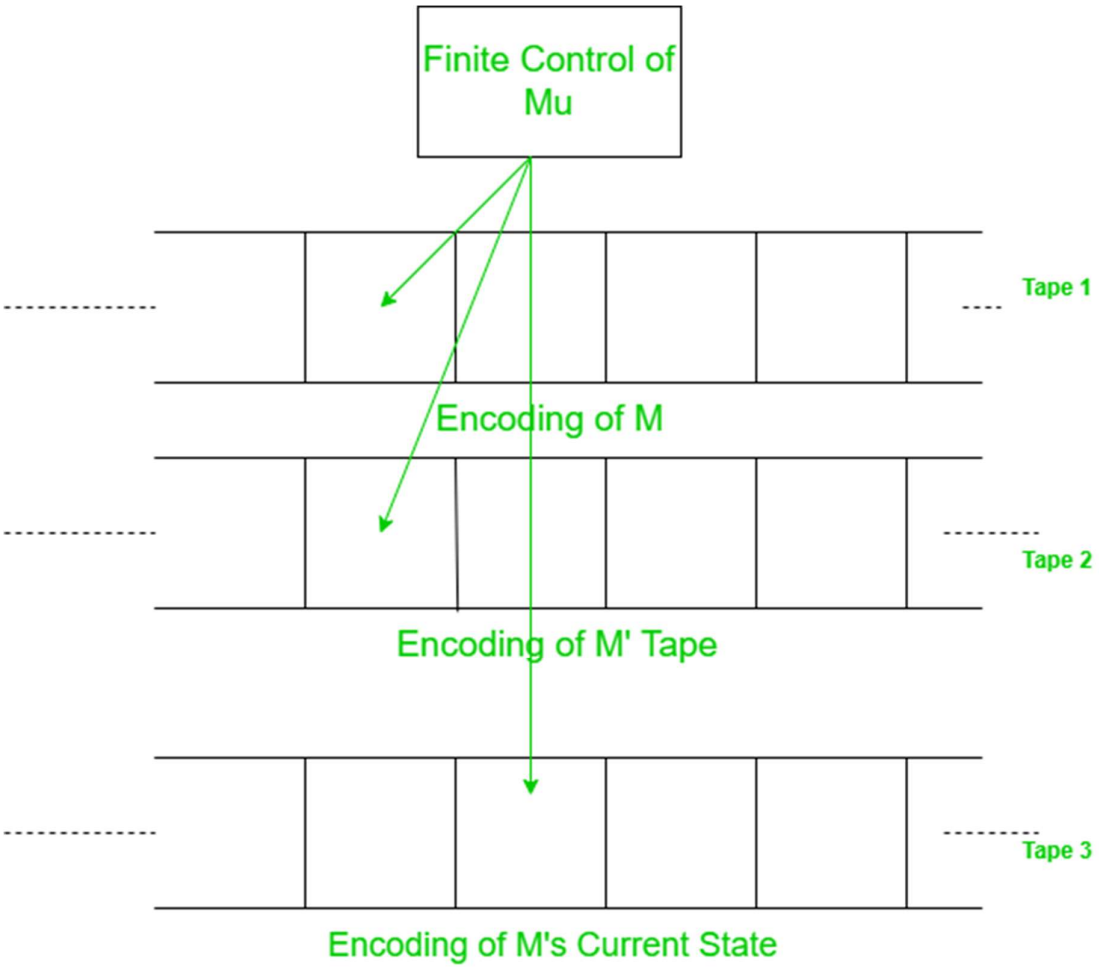
01110101111010111010

UTM Construction

Implementation of UTM

A UTM Mu then has an input alphabet = {0, 1} and the structure of a multi-tape TM.

- Mu looks first at the contents of Tape 2 and Tape 3 to determine the instantaneous description (ID) of M.
- It then consults Tape1 to see what M would do with this ID.
- Finally, Tape 2 and Tape 3 will be modified to reflect the result of the move.



UTM Implementation

If no transition for a given ID is formed, Mu halts as M must:

- In either case, Mu behaves as M would.
- If M halts, when presented with string w then Mu will halt when presented with the encoded M and the encoded string on its tape.
- Moreover, the final string Mu .s tape will be the encoding of the string.
- When M halts, Mu can tell if it is in the single accepting state and so moves to an accepting state of its own (or not).

Conclusion

The UTM in computer science was the conceptual basis, essentially a theoretical framework upon which many conceptions about the nature of computation were erected. This capability of the UTM to simulate any computable process underlines a universality principle, one that informs modern computer design and the study of algorithms and complexity theory. As a matter of fact, even though technology has

advanced, UTM serves to this day as the basis for modes of investigation into the limits of computation into areas such as artificial intelligence and machine learning, among others. This has remained relevant, shaping our understanding of current and future computational systems.

Difference Between Turing Machine and Universal Turing Machine

Turing machines as well as the Universal Turing machines are essential theories that assist individuals learn the basic operations of computers. In general, a Turing machine is a simple method of demonstrating how a machine can work according to certain instructions to solve mathematical problems or sort data, for instance. On the other hand, there exists a Universal Turing machine to simulate any other Turing machine thus implying the possibility of the former to follow instructions of the latter and consequently solve any problem solvable by those machines. This article is going to tell what both do, how they differ, and how they are significant to computing science.

What is a Turing Machine?

The Turing Machine was first described by Alan Turing in the year 1936. It was primarily invented to investigate the computability of a given problem. It accepts type-0 grammar which is Recursively Enumerable language. The Turing machine has a tape of infinite length where we can perform read and write operations. The infinite cells of the Turing machine can contain input symbols and blanks. It has a head pointer that can move in any direction, it points to the cell where the input is being read.

Advantages of a Turing Machine

- **Simplicity:** Turing machines also play a very important role in conceptualizing computational devices as they are rather simple but rather effective.
- **Flexibility:** They could mimic any existing algorithm and it would take them a little while to pull off the stunt, that is depending on the available resources.

Disadvantages of a Turing Machine

- **Limited Scope:** This explains why a standard Turing machine is limited in its capacity to perform any task outside the one for which it was configured.
- **Not Practical:** Although they are powerful tools for computation, Turing machines are not physically realizable, and thus not suitable for realizable computing pursuits.

What is a Universal Turing Machine?

Universal Turing Machine simulates a Turing Machine. Universal Turing Machine can be considered as a subset of all the Turing machines, it can match or surpass other Turing machines including itself. Universal Turing Machine is like a single Turing Machine that has a solution to all problems that is computable. It contains a Turing Machine description as input along with an input string, runs the Turing Machine on the input and returns a result.

Advantages of a Universal Turing Machine

- **Versatility:** A UTM has the ability to mimic any Turing machine, and hence UTM is very powerful in theory.
- **Foundation for Modern Computers:** Currently, UTM is the basis of modern general-purpose computer.

Disadvantages of a Universal Turing Machine

- **Complexity:** In contrast to a STTM, a UTM is more difficult to comprehend and to analyze.
- **Theoretical Nature:** UTMs are also non-constructive and therefore cannot be implemented in reality as with Turing machines.

Difference Between Turing Machine and Universal Turing Machine

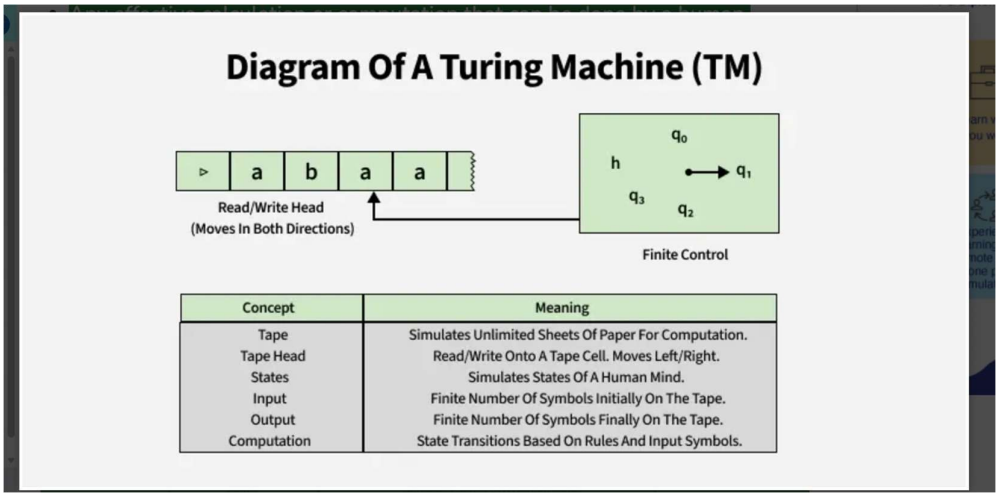
Turing Machine	Universal Turing Machine
It is a mathematical model of computation it manipulates symbols on the tape according to the rules defined	Universal Turing Machine is like a single Turing Machine that has a solution to all problem that is computable
A program can be compared to a Turing Machine	Programmable Turing Machine is called Universal Turing Machine
Turing machine's temporary storage is tape. The infinite cells of the Turing machine can contain input symbols and blanks.	Universal Turing Machine contains Turing Machine description as input along with an input string, runs the Turing Machine on the input and returns the result.

Turing Machine	Universal Turing Machine
Turing machines help us understand the fundamental limitations of mechanical computation power	Although developed for theoretical reasons, it helped in the development of stored program computers
A Turing machine is a formal model of a computer with a fixed program	Universal Turing Machine provides a solution to problems that are computable
It does not minimize the space complexity	It minimizes space complexity
Transition function which Turing Machine performs is defined as: δ X T -> Q X T X {L,R}, where δ is the transition function	The transition function is Q × T → Q × T × {L, R}, where Q is a finite set of states, T is the tape of the alphabet
In the set theory point of view, all Turing machines form a set of the all the device that accepts type 0 grammar	Universal Turing Machine is a subset of all the Turing Machines

Church's Thesis for Turing Machine

The Church-Turing Thesis is an important idea in the study of computability i.e. the ability to solve problems using a set of rules or procedures. It is an abstract model of a computing device, proposed by **Alan Turing and Alonzo Church**. It helps define algorithms and computing processes.

The basic idea of the thesis is:

- Any effective calculation or computation that can be done by a human (following specific steps) can also be done by a Turing machine.
- A Turing machine is a simple mathematical model that represents a basic form of a computer. This machine helps explain the logic behind modern computers.
- 

Turing Machine

To define these algorithms clearly, Alonzo Church developed a method called "M" for manipulating strings using logic and mathematics. This method must meet the following criteria:

1. **Finite instructions**: The method must have a limited number of steps.
2. **Finite output**: The method should produce a result after a certain number of steps.
3. **Real-life feasibility**: The method should be physically possible.
4. **Simple to understand**: It should not require complex understanding.

Based on these conditions, Church proposed the Church-Turing Thesis, which states:

"Every computation that can be done in the real world can be effectively performed by a Turing machine."

This idea was first formulated by Church in 1930 and is known as the Church-Turing Thesis. Although it cannot be proven, the hypothesis assumes that all computable functions can be represented by partial recursive functions.

In simpler terms:

- **Assumption 1**: Every function must be computable.
- **Assumption 2**: If a function **F** is computable and you perform basic operations on it to get a new function **G**, then **G** is also computable.

Importance of the Church-Turing Thesis

- **Defines computability**: The Church-Turing Thesis provides a clear definition of what is considered "computable" in computer science.
- **Standard for algorithms**: It defines "algorithmically computable" as anything that can be computed by a Turing machine.
- **Basis for computability**: It helps in understanding which problems can be solved using computers.
- **Foundation of computer science**: The thesis is fundamental to the study of **computability** and sets the foundation for modern computer science.

Relationship Between Turing Machines and Lambda Calculus

Equivalent Power

Though they look very different, Turing Machines and Lambda Calculus are equivalent in terms of computational power. This means:

- Anything you can compute with a Turing Machine can be computed using Lambda Calculus.
- Anything you can compute with Lambda Calculus can be simulated by a Turing Machine.

This equivalence was crucial to supporting Church's Thesis because it showed two very different formal systems capture the same notion of "computable function."

Different Perspectives

- **Turing Machines** focus on a mechanical step-by-step process involving states and symbols on a tape.
- **Lambda Calculus** focuses on symbolic manipulation of functions and substitution.

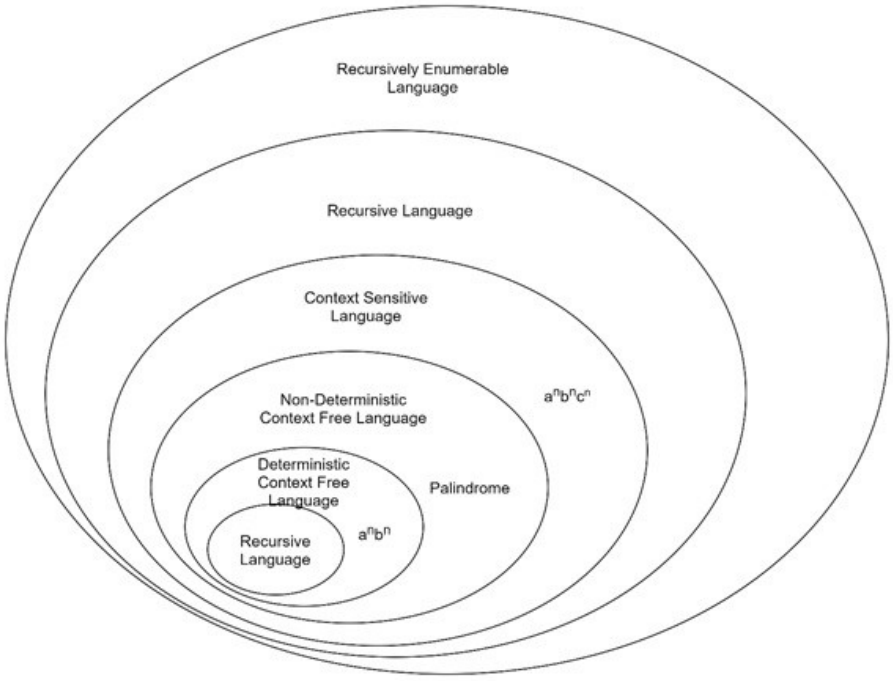
Recursive, and recursively enumerable languages:

Recursively Enumerable (RE) Language is an interesting and important concept in Automata Theory. In this chapter, we will provide an overview on the basics of Recursively Enumerable languages, and their properties and also see why we need this in theoretical computer science.

Recursively Enumerable Languages

In simple words, a "language" is a collection of strings, like words in a dictionary. A recursively enumerable language is a language where we can create a computer program (or a Turing machine) that can systematically list out all the strings that belong to the language.

Consider a machine that can generate all the possible sentences in the English language, one by one. This machine wouldn't necessarily know which sentences are not in the English language, but it could list out all the valid sentences. This is the idea of a recursively enumerable language. It can enumerate all the strings that are part of the language.



Turing Machines and Its Acceptance

The Turing machine can be used in defining Recursively Enumerable languages. It is a theoretical model that can perform basic operations like reading, writing, and moving

on a tape. A Turing machine can be programmed to accept or reject strings based on certain rules.

A Turing machine recognizes a language if, for every string that belongs to the language, the machine eventually enters a special "accept" state. For strings that are not in the language, the machine might either enter a "reject" state or continue running indefinitely without stopping.

A Turing machine can loop forever on strings that are not part of the language. This means it might never stop processing, never reaching a "reject" state. This unique characteristic is a defining feature of RE languages.

Recursive Languages: A Subset of RE Languages

Another important subset of RE languages is recursive language. In a recursive language, the Turing machine not only accepts strings belonging to the language but also always halts for strings that are not in the language.

For instance, consider a machine that can recognize all the valid sentences in the English language and, for any string that is not a sentence, will say "This is not a sentence" and then stop. This machine would be capable of recognizing a recursive language.

As an example, Consider the language L={anbncn|n≥0}. This language consists of strings where the number of a's, b's, and c's are equal.

- **RE Language** – We can build a Turing machine that starts at the beginning of the string and systematically checks if the number of 'a's, 'b's, and 'c's are equal. If they are, it accepts the string. However, if the string is not of this form, the machine may never halt, potentially looping forever. This makes L a recursively enumerable language.
- **Recursive Language** – We can also construct a Turing machine that checks if the number of 'a's, 'b's, and 'c's are equal. If they are, it accepts the string. If they are not, it reaches a "reject" state and halts. This makes L a recursive language as well.

Closure Properties of Recursive Languages

Recursive languages possess an interesting property called closure. This means that certain operations performed on recursive languages result in another recursive language.

Here are some key closure properties –

Union

If L1 and L2 are two recursive languages, their union (L1 ∪ L2) is also recursive. Imagine a machine that checks if a string belongs to L1 or L2; if it does, it accepts. Since both machines for L1 and L2 will eventually halt, this combined machine will also halt, making the union recursive.

Concatenation

If L1 and L2 are two recursive languages, their concatenation (L1.L2) is also recursive. Imagine a machine that first checks if the first part of the string belongs to L1, and if it does, it checks the remaining part of the string for L2. Since both L1 and L2 machines halt, this combined machine will also halt.

Kleen Closure

If L1 is a recursive language, its Kleen closure (L1*) is also recursive. This means the language including all possible combinations of strings from L1 concatenated together, including the empty string, is also recursive.

Intersection

If L1 and L2 are two recursive languages, their intersection (L1 ∩ L2) is also recursive. Imagine a machine that checks if a string belongs to both L1 and L2. Since both L1 and L2 machines halt, this combined machine will also halt.

Complement

If L1 is a recursive language, its complement (L1') is also recursive. This means the language containing all strings *not* in L1 is also recursive.

Conclusion

In this chapter, we presented a basic overview of recursively enumerable languages which gives a powerful framework for understanding what can be computed by machines. We covered their properties, examples, and closure properties as well, for a clear understanding of these concepts.

2. Deep Difference Table (TOC Style)

Feature	Recursive Languages (R / Decidable)	Recursively Enumerable Languages (RE / Semi-decidable)
TM behavior on strings IN the language	TM **halts & accepts**	TM **halts & accepts**
TM behavior on strings NOT IN the language	TM **halts & rejects**	TM may **reject OR loop forever** (no guarantee of halt)
Halting guarantee	Always halts for all inputs	Halts only for accepted strings; may not halt for others
Decision power	**Decidable** (gives YES/NO)	**Recognizable** (gives YES, but NO may be infinite loop)
Algorithm exists?	Yes, a total algorithm exists	Only a partial algorithm exists
Complement closure	Closed under complement	NOT closed under complement
Union	Closed	Closed
Intersection	Closed	Not always closed
Difference	Closed	Not closed
Kleene Star	Closed	Closed
Subset relation	**All recursive ⊆ RE**	RE has languages not in Recursive
Decider vs Recognizer	Language has a **TM Decider**	Language has a **TM Recognizer**
Output guarantee	Always outputs TRUE/FALSE	Guaranteed TRUE only; FALSE may run forever
Examples	Regular languages, CFLs, simple arithmetic languages	Halting problem language, Post Correspondence Problem language
Real intuition	“TM always answers.”	“TM only answers YES.”

Halting Problem in Theory of Computation

The halting problem is a fundamental issue in theory and computation. The problem is to determine whether a computer program will halt or run forever.

The **Halting Problem** in *Theory of Computation* is the challenge of determining whether a given program will **terminate** or run **forever** for a specific input. Formally, it asks: *"Given a program P and input I, can we design an algorithm that decides if P will halt on I?"*

Alan Turing proved in 1936 that **no general algorithm** can solve this for all possible programs and inputs — making it an **undecidable problem**. This means the only guaranteed way to know if a program halts is to run it and observe.

Key Concepts

- **Decidability**: A problem is decidable if there exists an algorithm that always produces a correct yes/no answer.
- **Undecidability**: No such algorithm exists for all cases. The Halting Problem falls here.
- **Turing Machine**: A mathematical model of computation used to formalize the problem.

Definition: The Halting Problem asks whether a given program or algorithm will eventually halt (terminate) or continue running indefinitely for a particular input. "Halting" means that the program will either accept or reject the input and then terminate, rather than going into an infinite loop.

The Core Question: Can we create an algorithm that determines whether any given program will halt for a specific input?

Answer: No, it is impossible to design a generalized algorithm that can accurately determine whether any arbitrary program will halt. The only way to know if a specific program halts is to run it and observe the outcome. This makes the Halting Problem an **undecidable problem**.

We can refrain the halting problem question in such a way also: Given a program written in any programming language (C/C++, Java, etc.), will it enter an infinite loop or will it terminate? This question cannot be answered by a general algorithm, making it undecidable.

To understand better the halting problem, we must know Decidability, Undecidability and Turing machine, decision problems and also a theory named as Computability theory and Computational complexity theory. Some important terms:

- **Computability theory -** The branch of theory of computation that studies which problems are computationally solvable using different model. In computer science, the computational complexity, or simply complexity of an algorithm is the amount of resources required for running it.
- **Decision problems -** A decision problem has only two possible outputs (yes or no) on any input. In computability theory and computational complexity theory, a decision problem is a problem that can be posed as a yes-no question of the input values. Like is there any solution to a particular problem? The answer would be either a yes or no. A decision problem is any arbitrary yes/no question on an infinite set of inputs.
- **Turing machine -** A Turing machine is a mathematical model of computation. A Turing machine is a general example of a CPU that controls all data manipulation done by a computer. Turing machine can be halting as well as non halting and it depends on algorithm and input associated with the algorithm.

Proof by Contradiction

Problem statement: Can we design a machine which if given a program can find out if that program will always halt or not halt on a particular input?

Solution:

1. Assumption: Suppose we can design such a machine, called **HM(P, I)**, where:

- **HM** is the hypothetical machine/program.
- **P** is the program.
- **I** is the input.

When provided with these inputs, HM will determine whether program P halts or not.

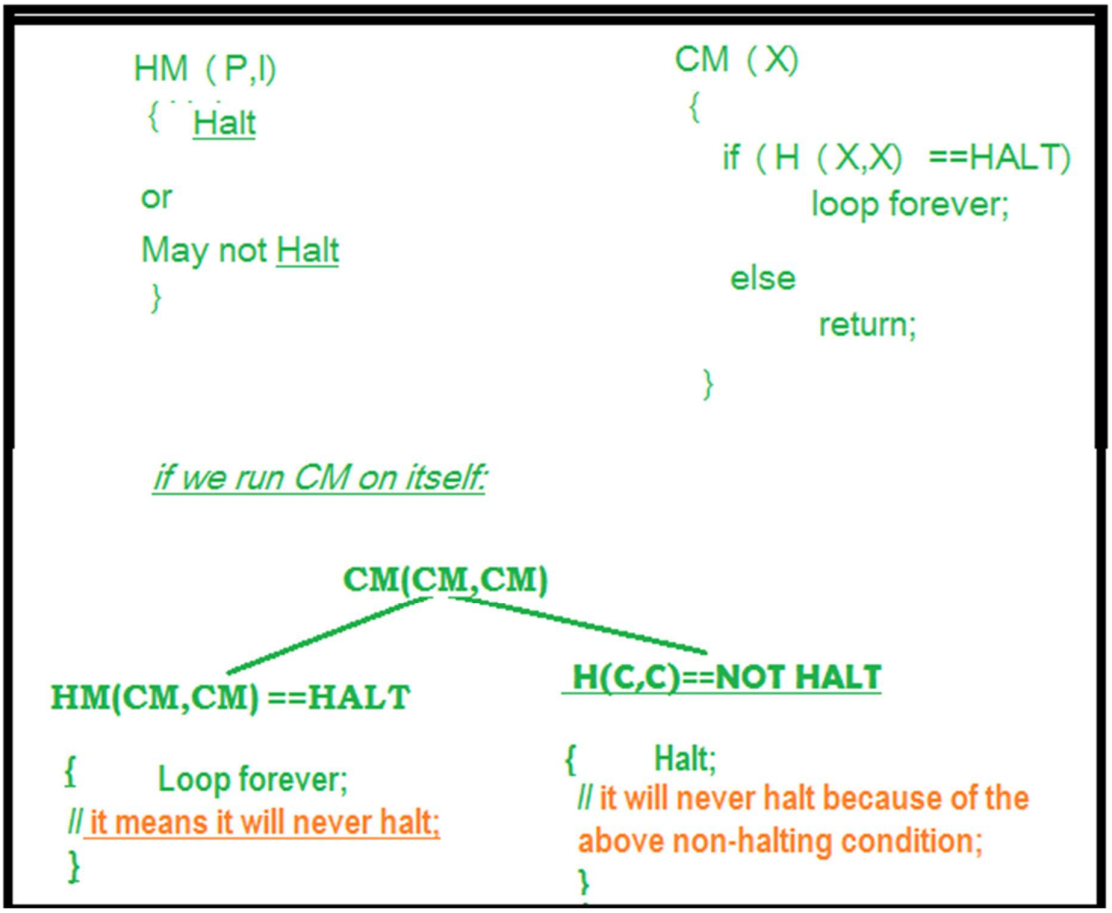
2. Constructing a Contradiction: Using HM, we can create another program, called **CM(X)**, where **X** is any program passed as an argument as shown in figure.

3. Behavior of CM(X): In CM(X), we call HM with inputs (X, X), meaning we pass program X both as the program and as its input. HM(X, X) will return either "Halt" or "Not Halt."

- If HM(X, X) predicts that X halts, CM(X) will enter an infinite loop.
- If HM(X, X) predicts that X does not halt, CM(X) will halt immediately.

4. The Contradiction: Now, consider what happens if we pass CM itself as an argument: **CM(CM)**.

- If HM predicts that CM(CM) halts, CM will loop indefinitely (contradicting HM's prediction).
- If HM predicts that CM(CM) does not halt, CM will halt immediately (again contradicting HM).



In both scenarios, we face a contradiction. Therefore, our initial assumption that such a machine HM could exist must be false. Hence we can conclude that Halting Problem is undecidable. No general algorithm can determine whether every program will halt or not.

Introduction to undecidability, undecidable problems about TM in TOC

Undecidability in the context of Turing machines (TMs) refers to the inability to find a Turing machine that can decide a specific problem in all cases. This means that there is no algorithm or computational model that can determine the answer to a problem with certainty.
Key points about undecidable problems include:

- **Undecidable Problems**: These problems cannot be solved by any Turing machine or algorithm. Examples include the Halting Problem and the Post Correspondence Problem.

- **Decidable Problems**: These problems can be solved by a Turing machine or algorithm. Examples include the equivalence of two regular languages and the finiteness of regular languages.

- **Reducibility**: A problem A is reducible to a problem B if there exists a function that converts strings in A to strings in B. If B is decidable, then A is also decidable. If B is undecidable, then A is also undecidable.

Understanding undecidability is crucial in the theory of computation, as it highlights the limitations of computational methods and the existence of problems that cannot be solved algorithmically

Decidable Problems A problem is decidable if we can construct a Turing machine which will halt in finite amount of time for every input and give answer as 'yes' or 'no'. A decidable problem has an algorithm to determine the answer for a given input. **Examples**

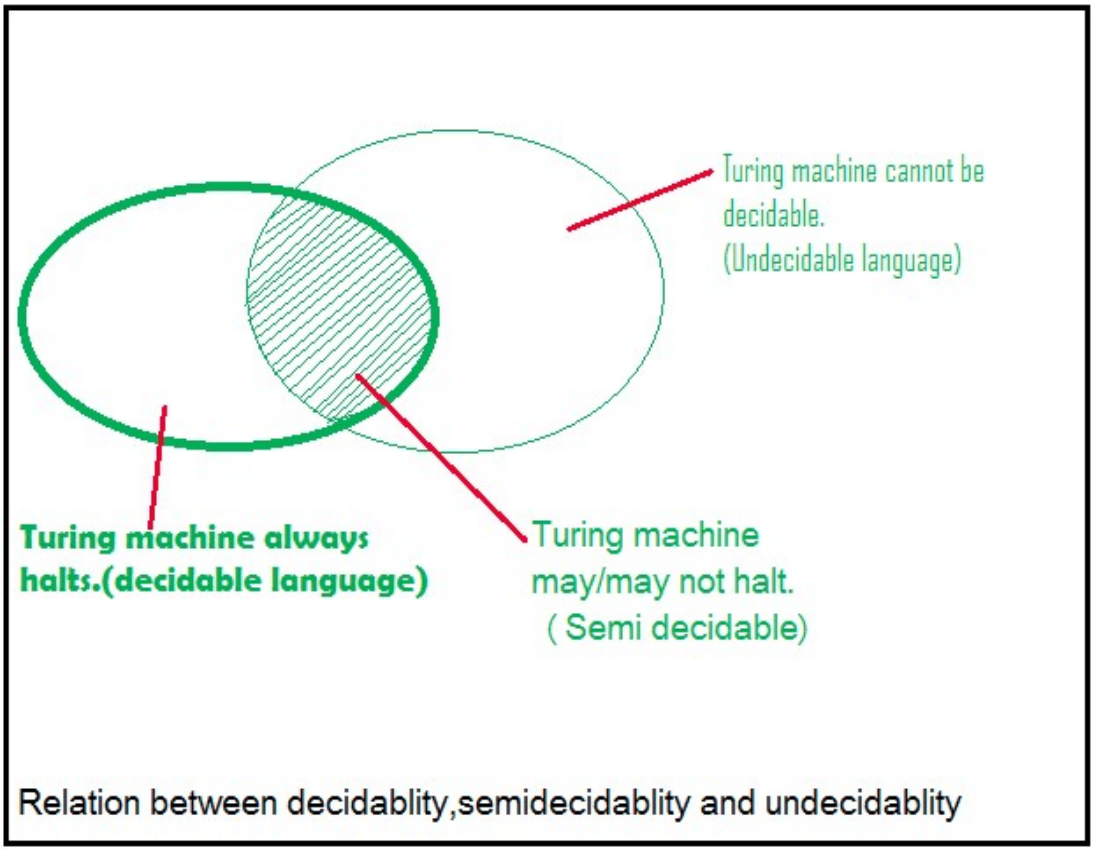
- **Equivalence of two regular languages:** Given two regular languages, there is an algorithm and Turing machine to decide whether two regular languages are equal or not.
- **Finiteness of regular language:** Given a regular language, there is an algorithm and Turing machine to decide whether regular language is finite or not.

- **Emptiness of context free language:** Given a context free language, there is an algorithm whether CFL is empty or not.

Undecidable Problems A problem is undecidable if there is no Turing machine which will always halt in finite amount of time to give answer as 'yes' or 'no'. An undecidable problem has no algorithm to determine the answer for a given input. **Examples**

- **Ambiguity of context-free languages:** Given a context-free language, there is no Turing machine which will always halt in finite amount of time and give answer whether language is ambiguous or not.
- **Equivalence of two context-free languages:** Given two context-free languages, there is no Turing machine which will always halt in finite amount of time and give answer whether two context free languages are equal or not.
- **Everything or completeness of CFG:** Given a CFG and input alphabet, whether CFG will generate all possible strings of input alphabet (?*)is undecidable.
- **Regularity of CFL, CSL, REC and REC:** Given a CFL, CSL, REC or REC, determining whether this language is regular is undecidable.

Note: Two popular undecidable problems are halting problem of TM and PCP (Post Correspondence Problem). Semi-decidable Problems A semi-decidable problem is subset of undecidable problems for which Turing machine will always halt in finite amount of time for answer as 'yes' and may or may not halt for answer as 'no'. Relationship between semi-decidable and decidable problem has been shown in Figure 1 as:



Relation between decidablity,semidecidablity and undecidablity

Rice's Theorem Every non-trivial (answer is not known) problem on Recursive Enumerable languages is undecidable.e.g.; If a language is Recursive Enumerable, its complement will be recursive enumerable or not is undecidable. **Reducibility and Undecidability** Language A is reducible to language B (represented as A?B) if there exists a function f which will convert strings in A to strings in B as:

w ? A <=> f(w) ? B

P, NP, CoNP, NP hard and NP complete | Complexity Classes

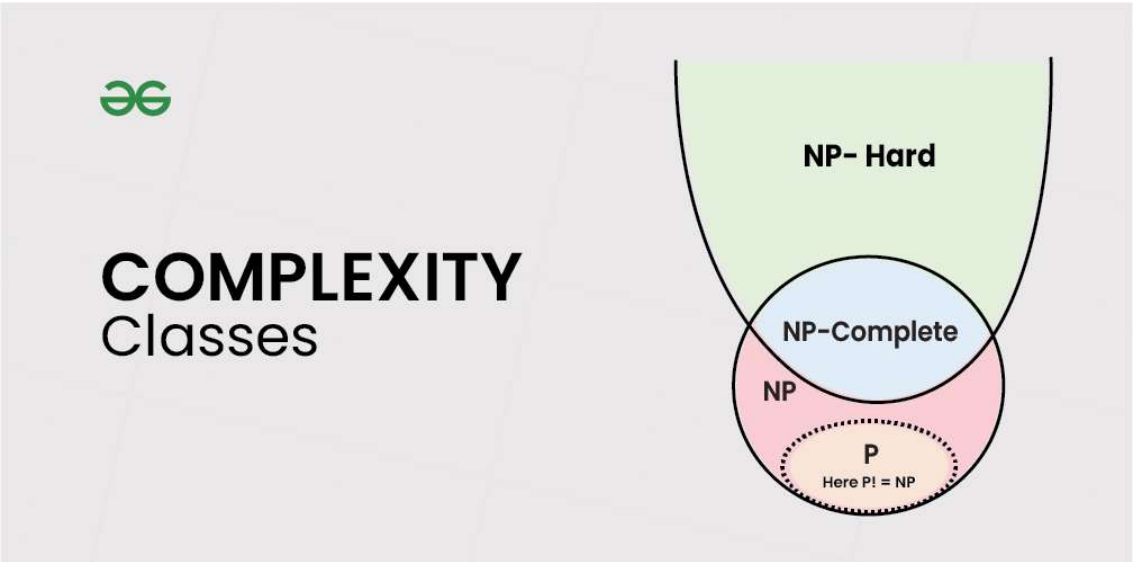
In computer science, problems are divided into classes known as **Complexity Classes**. In complexity theory, a Complexity Class is a set of problems with related complexity. With the help of complexity theory, we try to cover the following.

- Problems that cannot be solved by computers.
- Problems that can be efficiently solved (solved in Polynomial time) by computers.
- Problems for which no efficient solution (only exponential time algorithms) exist.

The common resources required by a solution are are time and space, meaning how much time the algorithm takes to solve a problem and the corresponding memory usage.

- The time complexity of an algorithm is used to describe the number of steps required to solve a problem, but it can also be used to describe how long it takes to verify the answer.
- The space complexity of an algorithm describes how much memory is required for the algorithm to operate.
- An algorithm having time complexity of the form O(nk) for input n and constant k is called polynomial time solution. These solutions scale well. On the other hand, time complexity of the form O(kn) is exponential time.

Complexity classes are useful in organizing similar types of problems.



Types of Complexity Classes

This article discusses the following complexity classes:

P Class

The P in the P class stands for **Polynomial Time.** It is the collection of decision problems(problems with a "yes" or "no" answer) that can be solved by a deterministic machine (our computers) in polynomial time.

Features:

- The solution to **P problem**s is easy to find.
- **P** is often a class of computational problems that are solvable and tractable. Tractable means that the problems can be solved in theory as well as in practice. But the problems that can be solved in theory but not in practice are known as intractable.

Most of the coding problems that we solve fall in this category like the below.

1. Calculating the greatest common divisor.
2. Finding a maximum matching.
3. Merge Sort

NP Class

The NP in NP class stands for **Non-deterministic Polynomial Time**. It is the collection of decision problems that can be solved by a non-deterministic machine (note that our computers are deterministic) in polynomial time.

Features:

- The solutions of the NP class might be hard to find since they are being solved by a non-deterministic machine but the solutions are easy to verify.
- Problems of NP can be verified by a deterministic machine in polynomial time.

Example:

Let us consider an example to better understand the **NP class**. Suppose there is a company having a total of **1000** employees having unique employee **IDs**. Assume that there are **200** rooms available for them. A selection of **200** employees must be paired together, but the CEO of the company has the data of some employees who can't work in the same room due to personal reasons.
This is an example of an **NP** problem. Since it is easy to check if the given choice of **200** employees proposed by a coworker is satisfactory or not i.e. no pair taken from the coworker list appears on the list given by the CEO. But generating such a list from scratch seems to be so hard as to be completely impractical.

It indicates that if someone can provide us with the solution to the problem, we can find the correct and incorrect pair in polynomial time. Thus for the **NP** class problem, the answer is possible, which can be calculated in polynomial time.

This class contains many problems that one would like to be able to solve effectively:

1. Boolean Satisfiability Problem (SAT).
2. Hamiltonian Path Problem.
3. Graph coloring.

Co-NP Class

Co-NP stands for the complement of NP Class. It means if the answer to a problem in Co-NP is No, then there is proof that can be checked in polynomial time.

Features:

- If a problem X is in NP, then its complement X' is also in CoNP.
- For an NP and CoNP problem, there is no need to verify all the answers at once in polynomial time, there is a need to verify only one particular answer "yes" or "no" in polynomial time for a problem to be in NP or CoNP.

Some example problems for CoNP are:

1. To check prime number.
2. Integer Factorization.

NP-hard class

An NP-hard problem is at least as hard as the hardest problem in NP and it is a class of problems such that every problem in NP reduces to NP-hard.

Features:

- All NP-hard problems are not in NP.
- It takes a long time to check them. This means if a solution for an NP-hard problem is given then it takes a long time to check whether it is right or not.
- A problem A is in NP-hard if, for every problem L in NP, there exists a polynomial-time reduction from L to A.

Some of the examples of problems in Np-hard are:

1. **Halting problem.**
2. **Qualified Boolean formulas.**

3. **No Hamiltonian cycle.**

NP-complete class

A problem is NP-complete if it is both NP and NP-hard. NP-complete problems are the hard problems in NP.

Features:

- NP-complete problems are special as any problem in NP class can be transformed or reduced into NP-complete problems in polynomial time.
- If one could solve an NP-complete problem in polynomial time, then one could also solve any NP problem in polynomial time.

Some example problems include:

1. Hamiltonian Cycle.
2. Satisfiability.
3. Vertex cover.

Complexity Class	Characteristic feature
P	Easily solvable in polynomial time.
NP	Yes, answers can be checked in polynomial time.
Co-NP	No, answers can be checked in polynomial time.
NP-hard	All NP-hard problems are not in NP and it takes a long time to check them.
NP-complete	A problem that is NP and NP-hard is NP-complete.

Master Table: P vs NP vs Co-NP vs NP-Hard vs NP-Complete

Class	Definition	Needs to be a Decision Problem?	Fast to Solve?	Fast to Verify?	Related To
P	Solvable in polynomial time using deterministic TM	Yes	✓ Yes	✓ Yes	"Easy problems"
NP	Verifiable in polynomial time	Yes	✗ unknown	✓ Yes	Nondeterminism
Co-NP	Complement problem is in NP	Yes	✗ unknown	✓ No-answers verifiable	Opposite of NP
NP-Hard	All NP problems reduce to it	No	✗ No	✗ Not required	Hardness class
NP-Complete	NP + NP-Hard	Yes	✗ No	✓ Yes	Hardest in NP

Difference between NP-Hard and NP-Complete:

NP-hard	NP-Complete
Polynomial time verification by a deterministic machine is not necessary.	Can be verified in Polynomial time by a deterministic machine.
NP-hard is not a decision problem.	NP-Complete is exclusively a decision problem.

NP-hard	NP-Complete
Not all NP-hard problems are NP-complete.	All NP-complete problems are NP-hard
Do not have to be a Decision problem.	It is exclusively a Decision problem.
Need not to be a NP problem	Must be NP
Example: Halting problem, Vertex cover problem, etc.	Example: Determine whether a graph has a Hamiltonian cycle, Determine whether a Boolean formula is satisfiable or not, Circuit-satisfiability problem and problems that are NP hard.

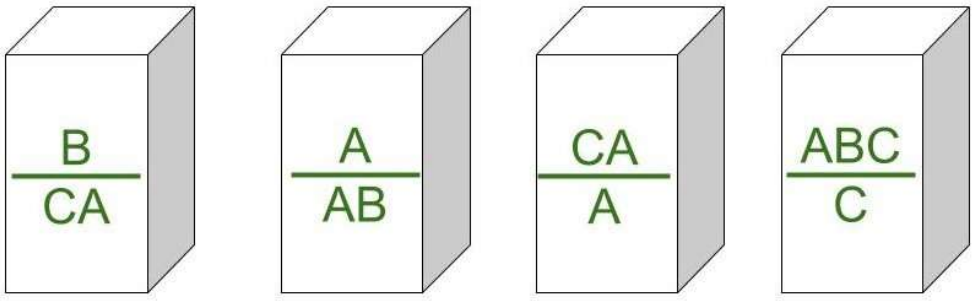
Post Correspondence Problem

Post Correspondence Problem is a popular undecidable problem that was introduced by Emil Leon Post in 1946. It is simpler than Halting Problem. In this problem we have N number of **Dominos** (tiles). The aim is to arrange tiles in such order that string made by Numerators is same as string made by Denominators. In simple words, lets assume we have two lists both containing N words, aim is to find out concatenation of these words in some sequence such that both lists yield same result. Let's try understanding this by taking two **lists A** and **B**

A=[aa, bb, abb] and B=[aab, ba, b]

Now for sequence 1, 2, 1, 3 first list will yield aabbaaabb and second list will yield same string aabbaaabb. So the solution to this PCP becomes 1, 2, 1, 3. Post Correspondence Problems can be represented in two ways:

1. Domino's Form :



2. Table Form :

	Numerator	Denominator
1	B	CA
2	A	AB
3	CA	A
4	ABC	C

Lets consider following examples. **Example-1:**

	A	B
1	1	111
2	10111	10
3	10	0

Explanation -

- **Step-1:** We will start with tile in which numerator and denominator are starting with same number, so we can start with either 1 or 2. Lets go with **second** tile, string made by numerator- 10111, string made by denominator is 10.
- **Step-2:** We need 1s in denominator to match 1s in numerator so we will go with **first** tile, string made by numerator is 10111 1, string made by denominator is 10 111.
- **Step-3:** There is extra 1 in numerator to match this 1 we will add **first** tile in sequence, string made by numerator is now 10111 1 1, string made by denominator is 10 111 111.
- **Step-4:** Now there is extra 1 in denominator to match it we will add **third** tile, string made by numerator is 10111 1 1 10, string made by denominator is 10 111 111 0.

Final Solution - 2 1 1 3

String made by numerators: 101111110

String made by denominators: 101111110

- As you can see, strings are same.

Example-2:

	A	B
1	100	1
2	0	100
3	1	00

Explanation -

- **Step-1:** We will start from tile **1** as it is our only option, string made by numerator is 100, string made by denominator is 1.
- **Step-2:** We have extra 00 in numerator, to balance this only way is to add tile **3** to sequence, string made by numerator is 100 1, string made by denominator is 1 00.
- **Step-3:** There is extra 1 in numerator to balance we can either add tile **1** or tile **2**. Lets try adding tile 1 first, string made by numerator is 100 1 100, string made by denominator is 1 00 1.
- **Step-4:** There is extra 100 in numerator, to balance this we can add **1st tile** again, string made by numerator is 100 1 100 100, string made by denominator is 1 00 1 1 1. The 6th digit in numerator string is 0 which is different from 6th digit in string made by denominator which is 1.

We can try unlimited combinations like one above but none of combination will lead us to solution, thus this problem does not have solution. **Undecidability of Post Correspondence Problem :** As theorem says that PCP is undecidable. That is, there is no particular algorithm that determines whether any Post Correspondence System has solution or not. **Proof -** We already know about undecidability of Turing Machine. If we are able to reduce Turing Machine to PCP then we will prove that PCP is undecidable as well. Consider Turing machine M to simulate PCP's input string w can be represented as

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$$

. If there is match in input string w, then Turing machine M halts in accepting state. This halting state of Turing machine is acceptance problem A_{TM}. We know that acceptance problem A_{TM} is undecidable. Therefore PCP problem is also undecidable. To force simulation of M, we make 2 modifications to Turing Machine M and one change to our PCP problem.

1. M on input w can never attempt to move tape head beyond left end of input tape.
2. If input is empty string € we use _ .
3. PCP problem starts match with first domino [u1/v1] This is called Modified PCP problem.

MPCP = {[D] | D is instance of PCP starts with first domino}

Construction Steps -

1. Put [# / (#q0w1w2..wn#)] into D as first domino, where is instance of D is MPCP. A partial match is obtained in first domino is # at one face is same #symbol in other face.
2. Transition functions for Turing Machine M can have moves Left L, Right R. For every x, y z in tape alphabets and q, r in Q where q is not equal to q_{reject}. If transition(q, x) = (r, y, R) put domino [qx / by] into D and transition(q, x) =(r, y, L) put domino [zqx / rzy] into D.
3. For every tape alphabet x put [x / x] into D.To mark separation of each configurations put [# / #] and [# / _#] into D .
4. To read input alphabets x even after Turing Machine is accepting state put [xqa / qa] and [qax / qa]and [qa# / #] into D. These steps concludes construction of D.

Recursive Function Theory in Theory of Computation (TOC)

Recursive function theory is a foundational concept in the **Theory of Computation (TOC)**, closely tied to Turing Machines and computability. It explores the mathematical framework for defining and analyzing functions that can be computed by algorithms, particularly those represented by Turing Machines.

Recursive and Partial Recursive Functions

A **recursive function** is a total function that can be computed by a Turing Machine that always halts. These functions are fully decidable, meaning the Turing Machine will determine an output for every valid input. In contrast, a **partial recursive function** may not halt for some inputs, making it semi-decidable.

For example, a Turing Machine (M) computes a function (f) such that for input (i_1, i_2, \dots, i_k), the output (f(i_1, i_2, \dots, i_k) = m) if the machine halts with (m) as the result. If the machine does not halt, (f) is undefined for that input.

Encoding Turing Machines as Integers

Turing Machines can be encoded as integers using binary representations. This encoding allows functions to map integers to Turing Machines and vice versa. For instance, the **index function** maps a Turing Machine to a unique positive integer, while the **reverse index function** maps integers back to Turing Machines. This encoding is crucial for defining recursive functions and analyzing their properties.

Key Theorems in Recursive Function Theory

1. **Smn Theorem**: This theorem states that for any partial recursive function (g(x, y)), there exists a total recursive function (s(x)) such that (f_{s(x)}(y) = g(x, y)). This enables the construction of specialized Turing Machines for specific inputs.
2. **Recursion Theorem**: This theorem guarantees the existence of a fixed point for any total recursive function. Specifically, for a function (f), there exists an (x_0) such that (f_{x_0}(x) = f(f_{x_0})(x)). This fixed-point property is fundamental in self-referential computations.

Recursive and Recursively Enumerable Languages

Recursive languages are those for which a Turing Machine always halts and decides membership. Recursively enumerable (RE) languages, on the other hand, are recognized by Turing Machines that may not halt for non-members. Recursive languages are a subset of RE languages.

For example:

- A recursive language (L) ensures the Turing Machine halts for all strings, either accepting or rejecting them.
- An RE language (L) ensures the Turing Machine halts and accepts strings in (L), but it may loop indefinitely for strings not in (L).

Closure Properties

Recursive languages are closed under operations like union, intersection, concatenation, and complementation. However, recursively enumerable languages are not closed under complementation, highlighting a key distinction between the two.

Applications

Recursive function theory underpins many concepts in TOC, such as the design of universal Turing Machines, the proof of undecidability (e.g., the Halting Problem), and the classification of languages. It also provides the mathematical tools to analyze the limits of computation and algorithmic processes